



260422 先行TTA 6本の処理フロー整理と案E''設計論点

◎ カテゴリ	設計
≡ キーワード	Energy OTTA SAL ゲーティング 論文執筆
📅 作成日	2026年4月22日

目的

- MI-IASW / BTTA-DG / T-TIME / BFT / T3A / SAR の test-time 処理フローを 1 枚で比較可能にする。
- 各手法の **適応媒体** / **backprop** 有無 / **gating** / **prototype** 扱い を明示し、案 E'' との差別化地点を可視化する。
- 案 E'' の実装前に詰めるべき 4 つの設計論点を記録する。

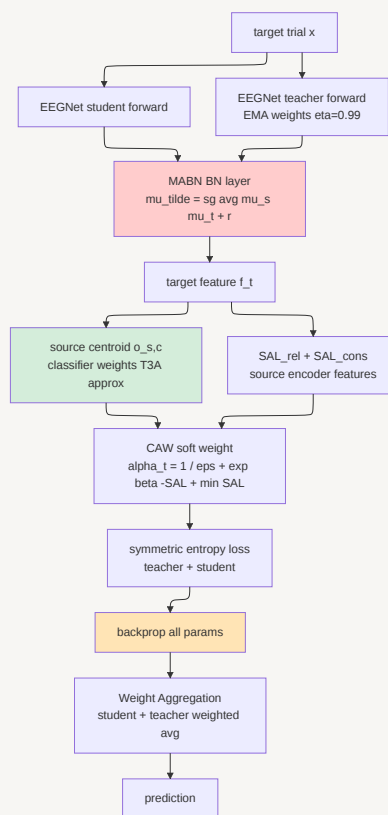
1. MI-IASW (Peng+ IJCNN 2025) — EEGNet + BN 適応系

📄 論文: [IEEE Xplore \(DOI: 10.1109/IJCNN64981.2025.11228030\)](https://doi.org/10.1109/IJCNN64981.2025.11228030)

```
[target trial x]
↓
EEGNet forward (student) + EEGNet teacher forward (EMA weights,  $\eta=0.99$ )
↓
BN 層で MABN:  $\mu \sim \text{sg}(\text{avg}(\mu_s, \mu_t)) + (\mu_t - \text{sg}(\mu_t))$ 
→ stat は mix、勾配は target 側のみ通す
↓
特徴  $f_t$  → source 側 class centroid  $o_{s,c}$  (classifier weights で近似、T3A 式)
↓
SAL_rel + SAL_cons を source encoder 特徴で計算
↓
CAW:  $\alpha_t^i = 1 / (\epsilon + \exp(\beta \cdot (-\text{SAL} + \min \text{SAL})))$  で per-sample 重み
↓
Symmetric entropy loss (teacher/student 両方向) × CAW 重み で backprop
↓
WA: student weights と teacher weights を加重平均
```

- 適応媒体: BN stat + 全 param
- Prototype: source 固定 (classifier weights 代用)
- Gating: CAW ソフト重み
- 結果: 2a 68.94% / 2b 84.25%

一行要約: 「BN 統計を source/target の平均で混ぜながら、teacher-student 構造で全 param を更新し、SAL ソフ



ト重みで混乱 trial の寄与を抜く」。

図で注目する点:

- 赤ノード MABN: BN stat に target 側の勾配を通していい。ここが **BN 凍結派の案 E'' との決定的な分かれ道**。
- 緑ノード o_s,c: prototype は source ままで固定。
「augmentation forward で prototype を robust 化」
や「EMA 更新」という案 E'' の做方と対極。
- CAW は soft weight なので、誤 **pseudo-label** も完全には止まらずじりじり混ぜる。案 E'' の Tri-Lock は hard abstain で止める設計。

案 E'' との距離:

- MI-IASW = 「BN 更新 + prototype 固定 + soft gate」
 - 案 E'' = 「BN 凍結 + prototype EMA + hard gate + 深度階層」
 - 直接競合の筆頭。案 E'' は論文の「我々の方法」表で、**MI-IASW と 4 軸すべてで相互排他に配置して新規性を立てる**。
- ▼  各ステップの詳細処理

2. BTTA-DG (Luo+ ICLR 2026) — LN-based SincAdaptNet, Gradient-free

 論文: [OpenReview \(ICLR 2026\)](#) / [PDF](#) / [GitHub \(コード\)](#).

[target trial x] → 1-48Hz bandpass + EA
↓
SincAdaptNet (LN, BN-free) forward → logits over T temporal steps
↓
post-softmax temporal embedding $X \in \mathbb{R}^{\{|L| \times T\}}$
↓
Dirichlet MLE: $\hat{\alpha} = \arg\max_{\alpha} \sum_j \log D(x_j; \alpha)$ ← projection $P: \mathbb{R}^{\{|L| \times T\}} \rightarrow \mathbb{R}^{\{|L|\}}$
↓
source 側で構築済みの GMM: $p(\alpha | y) = \sum_k \pi_{\{y, k\}} N(\alpha; \mu, \Sigma)$
↓
Bayesian 校正: $p_{\text{cal}}(y | \hat{\alpha}) = \frac{p_{\text{GMM}}(\hat{\alpha} | y) \cdot p_{\theta}(y)}{\sum p_{\text{GMM}}(\hat{\alpha} | y') \cdot p_{\theta}(y')}$
↓
conf > τ_{conf} かつ entropy < τ_{ent} なら memory bank に追加 → GMM 再推定

- 適応媒体: Dirichlet → GMM posterior
- Backprop: なし / BN: 構造的に回避 (LN 前提)
- 結果: 2a 78.70%

一行要約: 「BN を使わないために backbone ごと作り替え、post-softmax temporal 分布を Dirichlet で要約し、source 側で GMM を組んで Bayesian 校正する」。

図で注目する点:

- 緑ノード SincAdaptNet: 最初から LN-based、BN を避けるためにアーキテクチャ側を交換している。TCFormer/EEGNet には直接適用できない。
- 青ノード Dirichlet MLE: 時系列 logit 分布を $|L|$ -次元 concentration パラメータに投影。prototype とは別文脈の「確率分布表現」に凝2ている。
- 黄ノード Gate: conf AND entropy の 2 条件 hard filter。案 E'' の Tri-Lock (3 条件) の中間系列として位置づけられる。

案 E'' との距離:

- BTTA-DG = 「backbone 入れ替えて BN 問題を回避」
- 案 E'' = 「TCFormer (BN 入り) をそのままにして BN を凍結し、別媒体 (prototype) で適応」
- 方向性が直交しており、比較よりも補完関係。「BN を持つ既存 backbone に装着できる」点で案 E'' の実用性を主張できる。

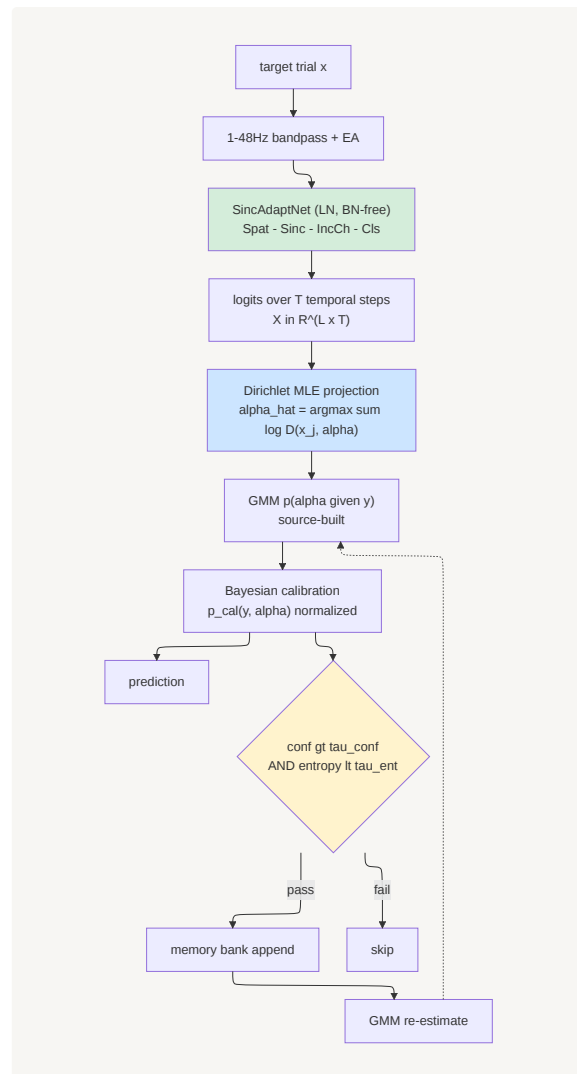
▼ 各ステップの詳細処理

Step 1: 前処理

1-48Hz bandpass フィルタ + EA (Euclidean Alignment, 全被験者共通の平均分散に正規化)。

Step 2: SincAdaptNet forward

Spat-Conv (空間フィルタ) → Sinc-Conv (学習可能帯域フィルタ) → IncCh-Conv (channel-wise) → Cls-



Conv。

すべて LN (Layer Normalization) で、BN は持たない。

Step 3: Temporal logits 取得

最終層で時系列 softmax 出力 $X \in \mathbb{R}^{|L| \times T}$ を得る。T は時刻方向の残存解像度。

Step 4: Dirichlet MLE projection

$$\hat{\alpha} = \operatorname{argmax}_{\alpha} \sum_j \log D(x_j; \alpha)$$

T 個の時刻 softmax ベクトルから $|L|$ -次元の Dirichlet concentration parameter を MLE で推定。時系列情報を 1 ベクトルに圧縮。

Step 5: Source-built GMM

train 時に class 条件付き GMM を構築: $p(\alpha | y) = \sum_k \pi_{\{y,k\}} \cdot N(\alpha; \mu_{\{y,k\}}, \Sigma_{\{y,k\}})$ 。

K 個の Gaussian 成分で class y を Dirichlet 空間上にモデル化。

Step 6: Bayesian 校正

$$p_{\text{cal}}(y | \hat{\alpha}) = p_{\text{GMM}}(\hat{\alpha} | y) \cdot p_{\theta}(y) / \sum_{y'} p_{\text{GMM}}(\hat{\alpha} | y') \cdot p_{\theta}(y')$$

先験 (source classifier 出力) と尤度 (GMM) を Bayes で統合。

Step 7: Memory bank 更新

$\text{conf} > \tau_{\text{conf}}$ かつ $\text{entropy} < \tau_{\text{ent}}$ を満たすサンプルのみをメモリに追加し、GMM を再推定 (online update)。

3. T-TIME (Li+ 2024) — EEGNet ensemble + 全 param 更新

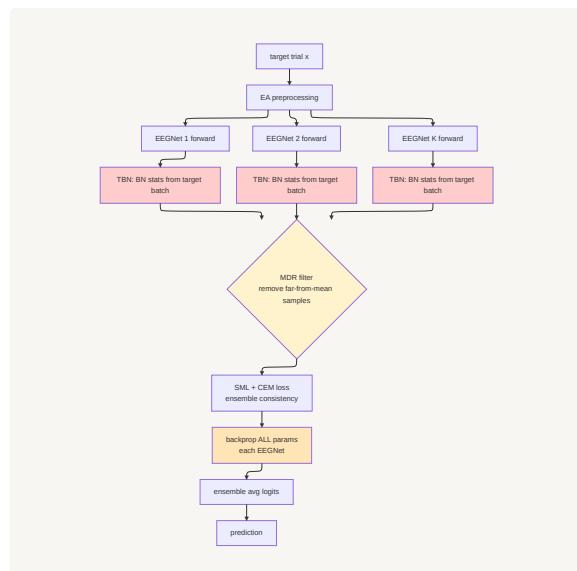
論文: [arXiv:2412.07228](https://arxiv.org/abs/2412.07228) / IEEE TPAMI

[target trial x] — EA 前処理
↓
K 個の EEGNet アンサンブル各自 forward
↓
TBN: target batch stat で BN 更新
↓
MDR (Mean-Distance Rejection): 過去特徴の平均から遠いサンプルを除外
↓
SML (Soft Margin Loss) + CEM (Consistency Ensemble Minimization) でアンサンブル一致損
↓
全 param **backprop** (Adam, lr 小)

- 適応媒体: BN + 全 param
- Prototype: なし / Gating: MDR
- 結果: 2a 79.30% (T3A は 68.23% で失敗報告あり)

一行要約: 「K 個の EEGNet を target batch stat で BN 更新し、互いに一致させるよう CEM/SML で全 param を backprop、外れ値は MDR で排除」。

図で注目する点:



- 赤ノード TBN が K 個並んでいる: **アンサンブル全員が BN を target 側に引きずり回される**。これが過去実験 S2 で含めた "BN 騒ぎ" の原因。
- オレンジ BP: 全 param を動かすため、latency も memory も最大級。オンライン BCI には不向き。
- MDR は「平均から遠い」だけで判定するため、**混乱 trial が中央がちなら通してしまうリスク**。

案 E'' との距離:

- T-TIME = 「重い (ensemble × 全 param) + BN 更新」
- 案 E'' = 「軽い (single forward + prototype のみ) + BN 凍結」
- **対比軸は「adaptation capacity」**。T-TIME は capacity を最大、案 E'' は最小。この間のスペクトル上での薬割りを議論できる。

▼ 各ステップの詳細処理

Step 1: 前処理

EA (Euclidean Alignment) により全被験者の共分散行列を正規化。

Step 2: K-EEGNet アンサンブル forward

K (≒5) 個の独立に初期化された EEGNet を並列に forward。各 EEGNet は独自の BN 統計を持つ。

Step 3: TBN (Target Batch Normalization)

BN 層では target batch から計算した stats をそのまま使う (source running stats を無視)。**各 EEGNet で BN を更新**。

Step 4: MDR (Mean-Distance Rejection)

過去に見た target feature の平均ベクトルを保持し、新サンプルの距離が閾値を超えたら outlier として除外。

Step 5: SML (Soft Margin Loss)

$$L_{SML} = -\sum p_i \log p_i^{\frac{1}{2}}$$

アンサンブル平均 pseudo-label p に対する柔らかい cross-entropy (label をそのままではなくスムージング)。

Step 6: CEM (Consistency Ensemble Minimization)

$$L_{CEM} = \sum_k KL(p_k || p_{avg})$$

K 個のアンサンブル間の予測の一致を促す。アンサンブルの多様性を潰さず、一致する場所だけ信じる。

Step 7: Backprop と aggregation

Adam (lr 小) で全 param を更新。最終予測は K-logit の平均 → argmax。

4. BFT (Li+ 2026) — Forward-only, logit aggregation

論文: [arXiv:2601.07556](https://arxiv.org/abs/2601.07556) / [HTML](#)

[target trial x]

↓

flowchart TD

X[target trial x] --> AUG[augmentatio

```

augmentation set {T_1, ..., T_M} を x に適用
↓
EEGNet forward × M 回 → logits_1, ..., logits_M (全て no_grad)
↓
learning-to-rank で信頼度の高い logit を選出
↓
選出済み logits を aggregation → 最終予測

```

- **適応媒体:** logit 集約のみ / モデル weight 不変
- **Prototype:** なし / **BN stats:** 更新なし
- **結果:** 2a 77.39%

一行要約: 「モデルを一切触らずに、入力側に M 通りの augmentation をかけて logit だけを選び・混ぜる」。

図で注目する点:

- 緑ノード × M: **全て no_grad / BN stats も不変**。最も安全 (fail-safe) な TTA。
- 黄ノード RANK: learning-to-rank は source 側で事前に学習済み。test 時には推論のみ。
- 図に横軸がない = 時間数的な適応がない。試行ごとに独立。つまり「時系列で何も持ち続けない」設計。

案 E'' との距離:

- BFT = 「不更新 (forward のみ)」
- 案 E'' = 「forward-only だが prototype を EMA で時系列的に更新」
- 案 E'' は BFT に「時間数」を追加した位置。BFT の安全性を残しつつ適応力を入れる方向。「source prototype の N=16 augmentation 集約」は BFT の test 時手法を **source 前に使っている** (他世代の流用)。

▼ 各ステップの詳細処理

Step 1: Augmentation 集合の準備

M 個 (例えば M=8) の augmentation 関数を用意。時間 shift, amplitude 変化, noise injection, channel dropout など。

source 側で augmentation の effectiveness は予め検証済み (**knowledge-guided augmentations**)。

Step 2: M-forward (no_grad)

各 $T_i(x)$ を EEGNet に no_grad で forward し、logits_i を得る。

モデル weight も BN stats も一切触らない (完全に forward-only)。

Step 3: Learning-to-rank

source 側で学習済みの ranker が各 logit の信頼度スコアを計算。理論的裏付け: pairwise rank loss (higher confidence ↔ correct prediction)。

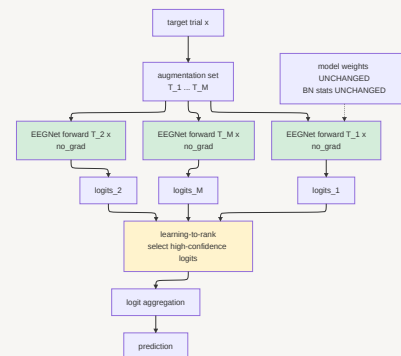
Step 4: 選出と aggregation

高スコアの logit のみ選び、加重平均で最終予測。モデルをそのまま、入力側を混ぜるだけの最も侵襲の少ない適応。

```

n set<br>T_1 ... T_M]
AUG --> F1[EEGNet forward T_1 x<br>no_grad]
AUG --> F2[EEGNet forward T_2 x<br>no_grad]
AUG --> FM[EEGNet forward T_M x<br>no_grad]
F1 --> L1[logits_1]
F2 --> L2[logits_2]
FM --> LM[logits_M]
L1 --> RANK[learning-to-rank<br>select high-confidence logits]
L2 --> RANK
LM --> RANK
RANK --> AGG[logit aggregation]
AGG --> OUT[prediction]
NOTE[model weights UNCHANGED<br>BN stats UNCHANGED] -.-> F1
style F1 fill:#d4edda
style F2 fill:#d4edda
style FM fill:#d4edda
style RANK fill:#fff3cd

```



Step 5: 不確定性抑制 (uncertainty suppression)

M 回の logit の分散が小さいと予測信頼度 高。
resource-constrained 現場 (脳波 wearable) にも移植可能。

5. T3A (Iwasawa+ NeurIPS 2021) — Classifier template 系

論文: [NeurIPS 2021 Proceedings](#) / [PDF](#)

```
[target batch x] - CV 想定、BN frozen
↓
f_θ(x) → 正規化特徴 z
↓
pseudo-label  $\hat{y} = \operatorname{argmax}_k q_\omega(y | z)$ 
↓
support set 更新:  $S^k \leftarrow S^k \cup \{z\}$  ( $y^k = \hat{y}$  の class)
↓
entropy top-M フィルタ:  $\alpha^k$  以下のみ残す
↓
centroid  $c^k = \operatorname{mean}(S^k)$  ← 新しい classifier weight
↓
予測 =  $\operatorname{argmax}_k \langle z, c^k \rangle$ 
```

- 適応媒体: classifier weight を cumulative set で置換
- Backprop: なし / EMA ではない (累積型)
- 結果: EEG MI で 68.23% < source 73.52% (T-TIME 報告)

一行要約: 「test 中に見た特徴を class ごとにただ累積し、その平均を classifier weight に差し替える」。

図で注目する点:

- 青ノード SUP: 「過去全部をびっしり残す」累積型。誤りが1個とってもずっと引きずる。EMA ではないことが重要。
- 黄ノード FILT: entropy top-M のみで選抜。SAL も Energy もないため、「低エントロピーな誤り」(自信ある誤分類)を通してしまう。
- EEG MI で失敗した理由はこの2つ: (i) 累積した誤りが消えない (ii) entropy 単独は EEG noise trial を止めにくい。

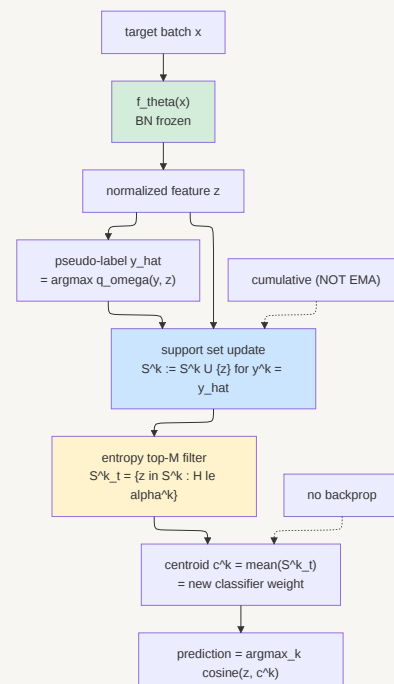
案 E'' との距離:

- T3A = 「累積型 prototype + entropy top-M」
- 案 E'' = 「EMA 型 prototype + Tri-Lock (pmax×SAL×Energy)」
- T3A の EEG MI 失敗 (T-TIME 報告 68.23%) は案 E'' の「累積と1条件 gate の組合せが悪い」という仮説を裏付ける負のエビデンスとして引用できる。

▼ 各ステップの詳細処理

Step 1: $f_\theta(x)$ 取得

```
flowchart TD
    X["target batch x"] --> FE["f_theta(x)  
BN frozen"]
    FE --> Z["normalized feature z"]
    Z --> PL["pseudo-label y_hat<br/>= argmax q_omega(y, z)"]
    Z --> SUP["support set update<br/>S^k := S^k U {z} for y^k = y_hat"]
    PL --> SUP
    SUP --> SUP
    SUP --> FILT["entropy top-M filter<br/>S^k_t = {z in S^k : H le alpha^k}"]
    FILT --> CEN["centroid c^k = mean(S^k_t)<br/>= new classifier weight"]
    CEN --> PRED["prediction = argmax_k cosine(z, c^k)"]
    NOTE1["cumulative (NOT EMA)"] -.-> SUP
    NOTE2["no backprop"] -.-> CEN
    style FE fill:#d4edda
    style SUP fill:#cce5ff
    style FILT fill:#fff3cd
    style PRED fill:#d4edda
```



pretrained encoder (BN frozen) で target batch を forward。BN も weight も一切触らない。

Step 2: Feature normalization

$z = f_{\theta}(x) / \|f_{\theta}(x)\|_2$ (L2 正規化)。cosine 類似度で
后で分類するため。

Step 3: Pseudo-label 生成

$\hat{y} = \operatorname{argmax}_y q_{\omega}(y | z)$ — 現在の classifier で予測
(これが template 更新されると変化)。

Step 4: Support set 更新 (累積型)

$S^k \leftarrow S^k \cup \{z\}$ for $y^k = \hat{y}$

累積型に全 z を残す — EMA ではない。一度 class k に
入った z は永久に残る (誤りが消えない → EEG MI 失敗
の元凶)。

Step 5: Entropy top-M filter

各 S^k 内で entropy H が大きい上位 M 件を除外。
 $S^k_t = \{z \in S^k : H(\hat{y} | z) \leq \alpha^k\}$ 。

閾値は entropy の M -位で動的に決まる ($M=20$ 付近)。

Step 6: Centroid 計算

$c^k = \operatorname{mean}(S^k_t)$ — 残った feature の単純平均が新
しい classifier weight。

Step 7: 予測

$\operatorname{predict} = \operatorname{argmax}_k \langle z, c^k \rangle$ (cosine 類似度)。

原著 classifier の weight は使わない — classifier を置
き換えた状態で推論。

6. SAR (Niu+ ICLR 2023) — GN/LN affine + SAM

論文: [arXiv:2302.12400](https://arxiv.org/abs/2302.12400) / [OpenReview \(ICLR 2023 Oral\)](#) / [GitHub \(コード\)](#)

```
[target batch x] — CV, GN/LN backbone
↓
forward → logits →  $E(x; \theta) = -\sum p \log p$ 
↓
reliable filter:  $S(x) = 1$  if  $E(x) < E_0 = 0.4 \cdot \ln(C)$ 
↓
sharpness-aware:  $\epsilon^{\wedge} = p \cdot \operatorname{sign}(\nabla E) \cdot |\nabla E| / \|\nabla E\|_2$ 
                   $E^{\wedge}SA = E(\theta + \epsilon^{\wedge})$ 
↓
min_0  $S(x) \cdot E^{\wedge}SA(x; \theta)$  で GN/LN affine
のみ backprop
↓
EMA entropy <  $e_0$  なら  $\theta$  をリセット (model
recovery)
```

- 適応媒体: GN/LN affine のみ
- BN を明示的に避ける: "the batch norm (BN) layer is a crucial factor hindering TTA stability"
- Prototype: なし

```
flowchart TD
    X["X[target batch x]"] --> FE["FE[forward GN/LN backbone]"]
    FE --> LOG["LOG[logits]"]
    LOG --> ENT["ENT[E x Theta = -sum p log p]"]
    ENT --> REL["REL{reliable filter<br/>E x < E_0 = 0.4 ln C}"]
    REL -->|fail| SKIP["SKIP[skip sample]"]
    REL -->|pass| SAM["SAM[sharpness-aware<br/>epsilon = rho sign grad E grad E / grad E_2]"]
    SAM --> ESA["ESA[E^SA = E Theta + epsilon]"]
    ESA --> OPT["OPT[min_Theta S x E^SA<br/>GN/LN affine ONLY]"]
    OPT --> REC["REC{EMA entropy < e_0?}"]
    REC -->|yes| RESET["RESET[reset Theta<br/>model recovery]"]
    REC -->|no| KEEP["KEEP[keep updated Theta]"]
    RESET --> FE
    KEEP --> FE
    NOTE["NOTE[BN explicitly avoided<br/>BN = instability source]"] --> FE
```

一行要約: 「BN が不安定源なので GN/LN で置き換え、その affine のみ SAM (平滑化) で更新、壊れたら巻き戻す」。

図で注目する点:

- 緑ノード FE: **BN を持たないモデルを前提とする**。これが SAR を EEG MI に直接使えない理由 (TCFormer/EEGNet は BN 入り)。
- オレンジ OPT: affine のみだが backprop する。案 E'' はこれすらやらない。
- 薄赤 REC: EMA entropy が壊れたらモデルリセット。案 E'' でも「**prototype が壊れたら source prototype に巻き戻す**」ロールバックを引用できる。
- **SAR のキー引用**: "BN is a crucial factor hindering TTA stability" は案 E'' の BN 凍結ドクトリンの **CV 独立ソースの根拠**。

案 E'' との距離:

- SAR = 「BN を GN/LN で置き換えて affine 更新」 (backbone 側で逃げる)
- 案 E'' = 「BN はそのまま凍結して prototype 側で適応」 (別経路を使う)
- **BN 問題へのアプローチが正反対**。SAR は「避ける」、案 E'' は「囲む」。

▼ 各ステップの詳細処理

Step 1: Forward (GN/LN backbone)

Group Norm または Layer Norm を持つ backbone (ResNet50-GN, ViT-LN) を前提とする。**BN を持たない**のが架の構築。

Step 2: Entropy 計算

$E(x; \Theta) = -\sum_i p_i \log p_i$ (softmax 出力 p の Shannon entropy)。

Step 3: Reliable filter

$S(x) = 1$ if $E(x) < E_0 = 0.4 \cdot \ln(C)$ else 0

C = class 数。 $\ln(C)$ は uniform distribution の entropy なので、**その 40% 以下**の丈け信頼できるサンプルを通す。

Step 4: SAM perturbation

$\hat{\epsilon} = \rho \cdot \text{sign}(\nabla_{\Theta} E) \cdot |\nabla_{\Theta} E| / \|\nabla_{\Theta} E\|_2$, $\rho=0.05$

entropy グラディエント方向に ρ 進んだ weight を考える (sharpness-aware)。

Step 5: E^SA 計算

$E^{\text{SA}}(x; \Theta) = E(x; \Theta + \hat{\epsilon})$

perturbed 時の entropy を loss に使う — 平滑な極小値に向かうよう誘導。

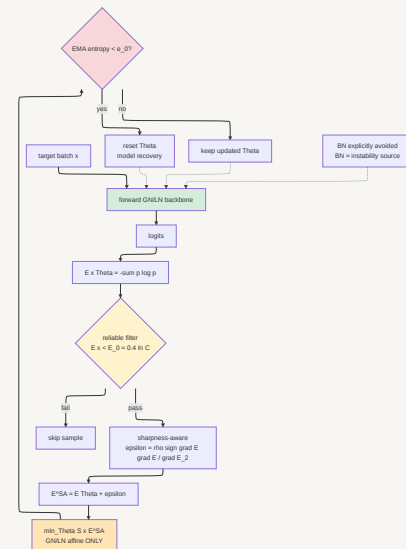
Step 6: Backprop (affine のみ)

$\min_{\Theta} S(x) \cdot E^{\text{SA}}(x; \Theta)$

GN/LN の affine パラメータ (α, β) のみ更新。running stats は持たないので stats は動かない。

Step 7: Model recovery

```
style FE fill:#d4edda
style OPT fill:#ffe4b5
style REL fill:#fff3cd
style REC fill:#f8d7da
```



EMA(entropy) < e_0 なら「モデルが壊れた」と判断し、 Θ を source 初期値にリセット。catastrophic failure からの自動復旧機構。

共通パターンと空白地帯

パターン	採用している先行
EA 前処理	BTТА-DG, T-TIME, BFT
Forward-only	BTТА-DG, BFT, T3A
BN 完全凍結	なし (SAR は GN/LN 前提、T3A は CV で frozen だが EEG MI では壊れる)
Prototype + EMA 更新	なし (T3A は累積、MI-IASW は source 固定)
Per-depth gating	なし (全て classifier 直前の単層)
3-way AND gate (conf × align × energy)	なし (MI-IASW は CAW ソフト重み、SAR/BTТА-DG は 2 条件)

現状最適: hybrid@0.01 (shallow_mean_deep_both)

📁 **Config:** [intentflow/offline/configs/tcformer_otta/tcformer_otta_bs1.yaml](#) / [Model](#)

```
[target trial x] - z-score (EA なし)
↓
TCFormer forward:
  MK-CNN → shallow_feat
  → Transformer (GQA) → intermediate_feat
  → TCN → deep_feat
  → classifier → logits
↓
Tri-Lock gate (single combined):
  pmax > 0.7 ∧ SAL > 0.5 ∧ Energy < 95th
↓
pass → BN stats update (momentum = 0.01):
  shallow BN: running_mean のみ更新 (var 凍結)
  deep BN: running_mean + running_var 両方更新
fail → BN 更新なし (forward は済み)
↓
prediction = argmax(logits)
```

- **適応媒体:** BN running stats (非均一 momentum)
- **Backprop:** なし
- **Gating:** Tri-Lock (pmax × SAL × Energy) single combined AND
- **現謎:** S2 で source より悪化 (-6.8pp)。BN 更新が深刻な被験者でドリフト源

一行要約: 「shallow は var を守り、deep だけ full 更新。Tri-Lock で誤 pseudo-label を変存させる最小侵襲 BN OTTA」。

図で注目する点:

- 赤ノード BN-deep: momentum=0.01 だが **var も更新される**。こが「BN 爆発」の潜在源。

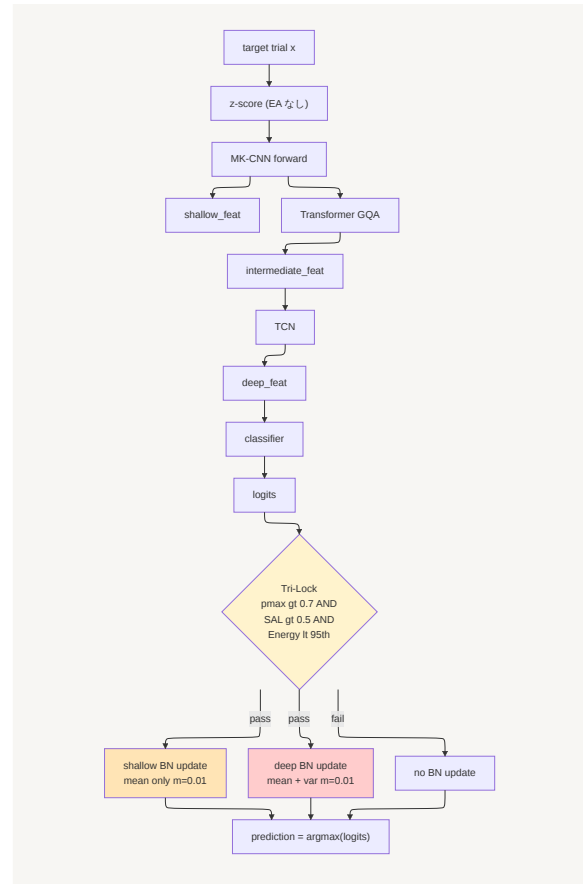
```
flowchart TD
    X["target trial x"] --> PRE["z-score (EA なし)"]
    PRE --> MKCNN["MK-CNN forward"]
    MKCNN --> SF["shallow_feat"]
    MKCNN --> TR["Transformer GQA"]
    TR --> MID["intermediate_feat"]
    MID --> TCN["TCN"]
    TCN --> DF["deep_feat"]
    DF --> CLS["classifier"]
    CLS --> LOG["logits"]
    LOG --> GATE["Tri-Lock<br/>pmax gt 0.7 AND<br/>SAL gt 0.5 AND<br/>Energy lt 95th"]
    GATE -- pass --> UPS["shallow BN update<br/>mean only m=0.01"]
    GATE -- pass --> UPD["deep BN update<br/>mean + var m=0.01"]
    GATE -- fail --> SKIP["no BN update"]
    UPS --> OUT["prediction = argmax(logits)"]
    UPD --> OUT
    SKIP --> OUT
    style UPS fill:#ffe4b5
    style UPD fill:#ffcccc
    style GATE fill:#fff3cd
```

- 黄ノード Gate: 3 条件 AND だが、**深刻な被験者 (S2 等)** では正誤分布が迷走した状態で **gate** を通かしてしまふ。これが案 E'' で hard abstain ドリフトを異質に切りたい理由。
- 黒構造: 現次元の比較軸にされている best design。案 E'' は「**BN を触らない** → **deep 更新を完全に外す** → **prototype 側に移す**」という組み目で差分を出す。

案 E'' との距離:

- 現 hybrid = BN を触る + prototype なし
- 案 E'' = BN を触らない + prototype 主體
- 同じ Tri-Lock 枠組みを持つが、**gate** を通したあとの「更新先」が正反対。

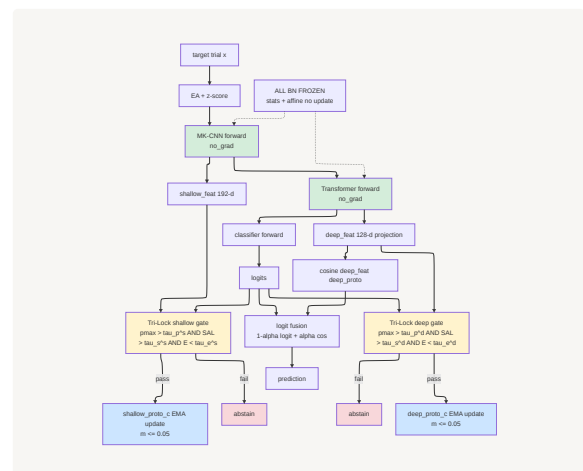
▼ 各ステップの詳細処理



案 E'' の処理フロー（対比のため再描画）

```

[target trial x] - EA + z-score
↓
TCFormer forward (no_grad):
  MK-CNN → shallow_feat (192-d)
  → Transformer → deep_feat (128-d projection)
  → classifier → logits
  ↓ 全 BN 凍結 (stats も affine も更新なし)
↓
Tri-Lock per-depth:
  shallow gate: pmax > τ_p^s ∧ SAL_shallow > τ_s^s ∧ Energy < τ_e^s
  deep gate: pmax > τ_p^d ∧ SAL_deep > τ_s^d ∧ Energy < τ_e^d
  ↓
  pass → proto_c ← (1-m) · proto_c + m · feat (m ≤ 0.05, per-depth 独立)
  fail → abstain (何も更新しない)
  ↓
  logit fusion: (1-α) · classifier_logit + α · cosine(deep_feat, deep_proto)
  
```



一行要約: 「BN を完全に凍らせたまま、shallow と deep の 2 層に置いた prototype を、per-depth Tri-Lock gate

を通った trial だけ EMA で少しずつ新しい被験者に寄せる」。

図で注目する点:

- 緑ノード (MKCNN, TR): **BN は全て凍結**。図に赤ノードが1つもない点が最大の記号 — 先行比較表で唯一「赤なし」の手法。
- 黄ノード (GATES, GATED): **shallow と deep で独立に判定**。shallow で reject でも deep で accept という非対称結果が起きる — 先行にない設計。
- 青ノード (UPDS, UPDD): **EMA だが per-depth 独立 momentum**。T3A の累積とも MI-IASW の source 固定とも違う。
- 薄赤 (ABS1, ABS2): **abstain が 2 系統ある**。safety 設計の高さ — オンライン BCI への移植を意識した構造。
- FUSE: logit 側で深さ d の prototype と混ぜる。alpha は論点 3。

4 つの新規性軸:

1. BN 完全凍結 (先行なし)
2. Shallow + Deep dual prototype (先行なし)
3. Per-depth Tri-Lock (先行なし)
4. Forward-only + prototype EMA と abstain の組合せ (先行なし)

▼  各ステップの詳細処理

案 F (対抗案 1): Logit-only Test-time Aggregation (LOTA)

コンセプト元: BFT + BT TA-DG の Bayesian calibration ハイブリッド。最軽量・全 forward-only・BN 完全凍結・prototype なし。

```
[target trial x] - EA + z-score
  ↓
augmentation set {T_1, ..., T_M} (M = 8)
  ↓
TCFormer forward × M (no_grad, BN 完全凍結):
  T_i(x) → logits_i
  ↓
Source-calibrated recalibration:
  logits_i' = logits_i - log p_source(y)
  ↓
Per-sample Tri-Lock (logit 空間):
  pmax > τ_p ∧ Energy < τ_e ∧ Var_aug(logits) < τ_v
  ↓
pass → logits = source-learned ranker weighted mean
fail → single-forward + abstain flag
  ↓
prediction
```

- 適応媒体: logit 空間のみ (aggregation + source prior recalibration)

```
flowchart TD
    X["target trial x"] --> PRE["EA + z-score"]
    PRE --> AUG["augmentation set T_1..T_M M=8"]
    AUG --> F1["TCFormer forward T_1 x<br/>no_grad, BN frozen"]
    AUG --> F2["TCFormer forward T_2 x<br/>no_grad, BN frozen"]
    AUG --> FM["TCFormer forward T_M x<br/>no_grad, BN frozen"]
    F1 --> L1["logits_1"]
    F2 --> L2["logits_2"]
    FM --> LM["logits_M"]
    L1 --> CAL["source-calibrated<br/>logit minus log p_source"]
    L2 --> CAL
    LM --> CAL
    CAL --> GATE["Tri-Lock logit-space<br/>pmax AND Energy AND Var_aug"]
    GATE -- pass --> AGG["ranker weighted mean"]
    GATE -- fail --> FB["single-forward +"]
```

- **Backprop:** なし / **BN:** 完全凍結 / **Prototype:** なし
- **Gating:** Tri-Lock ($p_{\max} \times \text{Energy} \times \text{augment-variance}$)。SAL は feature アクセスなしで成立しないため **Var_aug** に置換。

一行要約: 「M 回 augment forward の logit を source prior で校正し、ranker で重みづけて混ぜるだけ。モデルも BN もそのまま」。

案 E'' との違い:

- 案 E'' = 特徴空間で適応 (prototype EMA)
- 案 F = **logit 空間**で適応 (aggregation + calibration)
- **適応媒体が完全に直交**— どちらが EEG MI で有効かを直接比較できる。

メリット:

- 実装が最軽 (機能合成 10 時間以内)
- オンライン BCI への移植は自明 (forward-only, 時間数なし)
- どんな被験者でも source より壊れにくい (fail-safe)

デメリット:

- 適応容量が小さい— 2a/2b の長期 shift では適応不足になる可能性
- source prior が不正確な場合、recalibration が逆に有害
- BFT の提案コアと近い (新規性は source prior 校正部分のみ)

あり得る結果:

- source と同等か微小改善 (+1-2pp)。案 E'' の「下限線」として使える。

▼ 各ステップの詳細処理

Step 1: 前処理

EA + z-score。案 E'' と同一の前処理で揃え、比較の公平性を保つ。

Step 2: Augmentation 集合 (M=8)

source 側で選抜済みの M 個の変換 (時間 shift $\pm 10\text{ms}$, amplitude $\pm 10\%$, channel dropout $p=0.05$, Gaussian noise $\sigma=0.01$)。

Step 3: M-forward (no_grad, BN 完全凍結)

$T_i(x)$ を TCFormer に通し logits_i を得る。BN stats も affine も一切触らない (案 E'' と同様)。

Step 4: Source prior recalibration

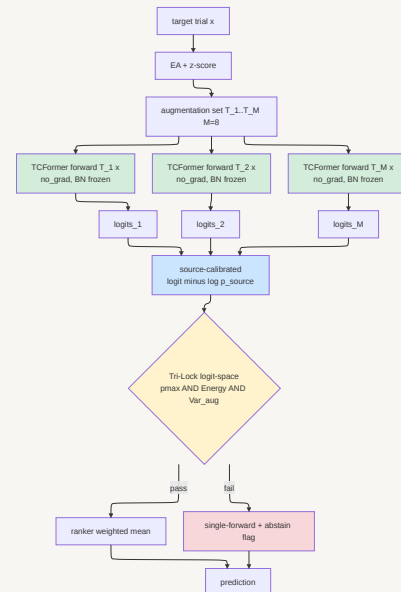
$\text{logits}_i' = \text{logits}_i - \log p_{\text{source}}(y)$

source train data の class frequency p_{source} を事前に計測しておき、target のクラス分布変化 (prior shift) を補正する。

Step 5: Tri-Lock (logit 空間)

- $p_{\max} > \tau_p$ (softmax 最大値)
- $\text{Energy} < \tau_e$ ($-\text{T-logsumexp}$)

```
abstain flag"]
AGG --> OUT["prediction"]
FB --> OUT
style F1 fill:#d4edda
style F2 fill:#d4edda
style FM fill:#d4edda
style CAL fill:#cce5ff
style GATE fill:#fff3cd
style FB fill:#f8d7da
```



- **Var_aug(logits_1,...,logits_M) < τ_v** — M 回
forward のバラつきが小さい

第 3 条件が案 E'' の SAL を置き換える (特徴アクセスなしに logit の安定性で判定)。

Step 6: Gate pass → Ranker weighted aggregation

$\text{final_logits} = \sum_i w_i \cdot \text{logits}_i$, $w_i = \text{ranker}(T_i, x)$

source 側で学習済みの ranker が変換-入力 pair ごとの重みを予測。

Step 7: Gate fail → single-forward + abstain

single $T_1(x)$ の logit をそのまま使う。abstain flag を立てて safety 系へ通知。

重要 property: 時系列的な適応がない (memory 不要)。各 trial が独立な横断面処理。

案 G (対抗案 2): Adapter-based Minimal PEFT-OTTA

コンセプト元: SAR + LoRA ハイブリッド。BN 凍結は保つが、Adapter (全 param の 1-2%) のみ backprop で更新。

```
[target trial x] → EA + z-score
↓
TCFormer (本体 frozen, Adapter だけ trainable):
  MK-CNN → shallow_feat
  → Transformer + Adapter_1 (dim 16) → intermediate_feat
  → TCN + Adapter_2 (dim 16) → deep_feat
  → classifier → logits
  ↓ (BN stats + main weights すべて凍結)
Tri-Lock gate (現構成と同一):
  pmax > 0.7 ∧ SAL > 0.5 ∧ Energy < 95th
  ↓
pass → entropy loss + SAM perturbation (p = 0.05):
  Adapter params のみ backprop
fail → skip
↓
Model recovery:
  EMA(entropy) < e_0 なら Adapter を source 初期化へ reset
  ↓
prediction
```

- **適応媒体:** Adapter (LoRA-like bottleneck) のみ
- **Backprop:** あり (Adapter のみ、全 param の 1-2%)
- **BN:** 完全凍結 / **Prototype:** なし
- **Gating:** Tri-Lock 現構成流用 + SAM (SAR) + Model recovery (SAR)

一行要約: 「TCFormer 本体と BN は凍結のまま、Transformer/TCN に挿した小さな Adapter だけを SAM で更新する」。

案 E'' との違い:

```
flowchart TD
    X["target trial x"] --> PRE["EA + z-score"]
    PRE --> MKCNN["MK-CNN forward<br/>frozen"]
    MKCNN --> TR["Transformer + Adapter_1<br/>LoRA dim 16"]
    TR --> TCN["TCN + Adapter_2<br/>LoRA dim 16"]
    TCN --> CLS["classifier frozen"]
    CLS --> LOG["logits"]
    NOTE1["ALL BN FROZEN<br/>main weights FROZEN"] -.-> MKCNN
    LOG --> GATE["Tri-Lock<br/>pmax AND SAL AND Energy"]
    GATE -- pass --> SAM["SAM perturbation<br/>rho = 0.05"]
    GATE -- fail --> SKIP["skip"]
    SAM --> BP["backprop Adapter ONLY<br/>>1-2 percent params"]
    BP --> REC["EMA entropy lt e_0 ?"]
    REC -- yes --> RESET["reset Adapter"]
    REC -- no --> KEEP["keep updated Adapter"]
    RESET -.-> TR
    KEEP -.-> TR
    LOG --> OUT["prediction"]
    style MKCNN fill:#d4edda
    style TR fill:#cce5ff
    style TCN fill:#cce5ff
    style BP fill:#ffe4b5
    style GATE fill:#fff3cd
    style RESET fill:#f8d7da
```

- 案 E'' = **forward-only / prototype** 媒体
- 案 G = **minimal backprop / Adapter** 媒体
- **BNを触らない点は共通だが、適応の「手法」が違う。**
案 E'' = EMA で template 的、案 G = backprop で検索的。

メリット:

- backprop の表現力を使いつつ BN 起因の不安定を回避
- SAR の SAM + Model recovery をそのまま移植可能 (先行の実績を流用)
- prototype の調整不要 (shallow/deep/EMA momentum/fusion alpha すべて不要)

デメリット:

- source 学習時に Adapter の挿入位置と dim を設計決定しておく必要 (re-train)
- forward-only でないためオンライン latency が案 E', F より大きい
- Adapter 差し引いても新規性は SAR の EEG MI 適用例 (先行なし だが approach は既知)

あり得る結果:

- hybrid を超えるか + 3~5pp。案 E'' と互角か、SAR の表現力で勝つ可能性

▼ 各ステップの詳細処理

Step 0: Source 再学習 (事前準備)

TCFormer の Transformer と TCN に LoRA-風 Adapter (bottleneck dim=16) を挿入し、source data で全体を再学習。

Adapter は初期状態では何もしない (residual がゼロ)。学習後の Adapter weight を source_adapter_0 として保存。

Step 1: 前処理

EA + z-score。案 E'' / F と同一。

Step 2: TCFormer 本体 + BN 凍結 forward

MK-CNN → Transformer (main weights frozen, Adapter_1 trainable) → TCN (main weights frozen, Adapter_2 trainable) → classifier (frozen) → logits。

全 BN は eval mode + momentum=0。

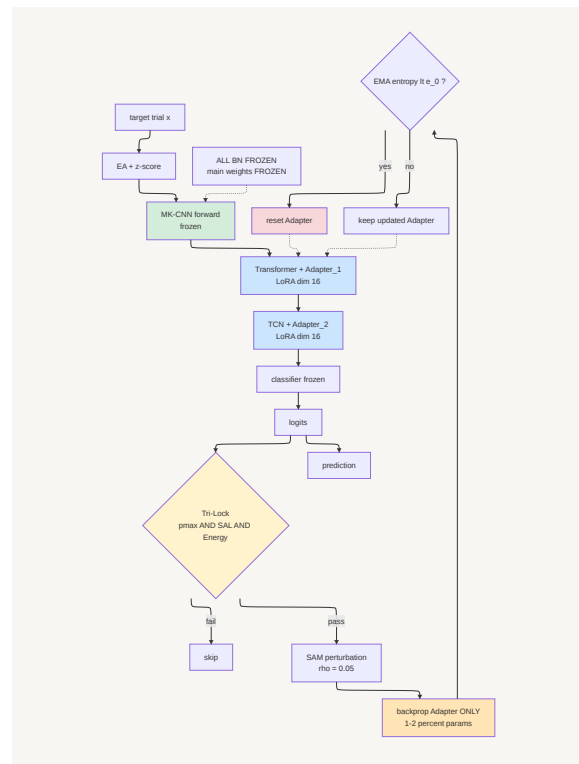
Step 3: Tri-Lock gate

$p_{max} > 0.7$ AND $SAL > 0.5$ AND $Energy < 95th$ (現 hybrid と同一構成。Adapter 側で適応するので gate は標準を使う)。

Step 4: Entropy + SAM perturbation

- $L_{entropy} = -\sum p \log p$ を Adapter のみに対して計算
- SAM: $\hat{\epsilon} = 0.05 \cdot \nabla_{\{\text{Adapter}\}} L / \|\nabla_{\{\text{Adapter}\}} L\|_2$
- $L^{SA} = L(\text{Adapter} + \hat{\epsilon})$ を最小化する勾配を取る

Step 5: Backprop (Adapter のみ)



Adam (lr=1e-4) で Adapter weights を 1-step 更新。
全 param の 1-2% のみ動くので BN も main weights も変化なし。

Step 6: Model recovery (SAR 流用)

EMA(L_entropy) を追跡し、それが e_0 を下回ったら Adapter を source_adapter_0 にリセット。崩壊からの自動復旧。

Step 7: 予測

predict = argmax(logits)。Adapter を含む forward は次の trial でも更新済み Adapter を使う (accumulation)。

被験者別: subject 開始時に Adapter を source_adapter_0 に reset (被験者間で Adapter が繋がらない)。

4 案の比較マトリクス

軸	現 hybrid@0.01	案 E'' (本命)	案 F (LOTA)	案 G (PEFT)	BN	更新あり (shallow mean / deep full)	完全凍結
Backprop	なし	なし	なし	Adapter のみ (1-2%)	適応媒体	BN running stats	Prototyp (shallow deep)
Gating	Tri-Lock single	Tri-Lock per-depth (非対称)	Tri-Lock logit-space (Var_aug 置換)	Tri-Lock single + SAM + Recovery	実装量	0 (実装済み)	~15h
オンライン適性	中 (BN 更新危険)	高 (forward-only + abstain)	最高 (fail-safe)	中 (backprop latency)	S2 悪化リスク	実測で発現済み	低 (BN 凍 abstain)
新規性	baseline	4 軸 OR 空白	source prior calibration (1 軸)	SAR の EEG MI 適用 (1 軸)	案 E'' との関係	比較元 (上限)	—

位置づけ:

- 現 hybrid = 比較の上限線 (既存ベスト)
- 案 F = 下限線 (何も変えない軽量型。source + 微調整程度を期待)
- 案 G = 案 E'' の「backprop 版」 (表現力で案 E'' と互角)
- 案 E'' = forward-only を保ちつつ適応力を中間に入れる

三つに根ろざさせることで「EEG MI OTTA で BN を触らない限り、何媒体が最も効いたか」という直接的な結論が得られる。

詰めたい設計論点 (4 点)

論点 1: Shallow prototype を初回から入れるか

- MI-IASW ablation が「prototype 単体は弱い」ことを示している。
- 案 A: 初回は deep-only で T3A の失敗と同じ土俵に立ち、shallow は ablation 軸に回す → 議論がクリーン
- 案 B: 最初から dual にし「hierarchy が効くか」を初回から示す → 新規性の見え方が強い
- 傾向: 案 A 推し (ablation 設計の健全性)

論点 2: Tri-Lock の第 3 条件 (Energy) は本当に必要か

- SAR は entropy のみ、MI-IASW は SAL のみ、BTDA-DG は entropy+conf の 2 条件。
- Energy を足す根拠は Liu+ 2020 の OOD 検出での有効性だが、EEG MI での実証はまだ薄い。

- 判断: preliminary で $p_{\max} \times \text{SAL}$ の 2 条件版と 3 条件版を比較してから決定する。

論点 3: Logit fusion の α

- $\alpha=0 \rightarrow$ prototype は「統計集めるだけで判定に効かない」(MI-IASW との差分が薄まる)
- $\alpha=1.0 \rightarrow$ T3A 式 (classifier 置換) で既に失敗報告あり
- 判断: $\alpha \in \{0.1, 0.3, 0.5\}$ の grid search を S5 で回す。

論点 4: Source prototype の augmentation forward N 回集約

- BFT は test 時に M 回 augment。案 E'' は **source 側のみ N=16 forward + mean** (robust 化目的)。
- 差別化のため「augmentation 集約は source のみ、test は single forward」を論文記述で明示する。
- BFT の test 時集約とは別文脈であることを強調。

色凡例 (Mermaid 図)

- ■ 赤 (ffcccc): BN 更新 (案 E'' が回避する点)
- ■ オレンジ (ffe4b5): backprop 更新
- ■ 緑 (d4edda): forward-only / BN 凍結
- ■ 青 (cce5ff): prototype / centroid 更新
- ■ 黄 (fff3cd): gating / filter
- ■ 薄赤 (f8d7da): abstain / reset

次アクション

- ☐ Codex による 案 E'' 実装レビューを待つ (test-time 側は Codex が実装中)
- ☐ 2b source model (87.74%) で first pass (source_only / vanilla-both / hybrid@0.01) を準備
- ☐ 論点 1-4 について、preliminary 結果が出た段階でゼミ相談
- ☐ MI-IASW 後続 + "BN-frozen prototype EMA" 類似研究を arxiv 監視

 [260425 Drift-Polarity / SafeCommit OTTA 設計メモ](#)